

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
BỘ MÔN PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH



PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH
ASSIGNMENT 04

Giảng viên hướng dẫn	: TRẦN ĐÌNH QUẾ
Họ và tên sinh viên	: NGUYỄN NGỌC HOÀNG NAM
Mã sinh viên	: B23DCCN585
Lớp	: D23CQCN11-B
Nhóm	: 05

Hà Nội – 2026

LỜI MỞ ĐẦU

Báo cáo trình bày quá trình tìm hiểu mạng nơ-ron tích chập (CNN), xây dựng mô hình cơ bản và mô hình cải tiến, sau đó cài đặt bằng ba cách: NumPy từ đầu, PyTorch và TensorFlow. Mục tiêu là hiểu cách các phép biến đổi hợp thành một mô hình học được, kiểm chứng lan truyền ngược và thực hiện một quy trình đánh giá có thể tái lập.

Phạm vi được chốt theo ảnh yêu cầu Assignment 04: ba dataset từ Kaggle gồm MNIST và hai bộ dữ liệu ảnh phức tạp hơn là CIFAR-10, CIFAR-100. Mỗi bộ có hai kiến trúc ở ba cách triển khai, tạo thành 18 thí nghiệm chính. Các bộ diabetes, nhà ở và đánh giá quần áo trong trao đổi ban đầu chưa thuộc phạm vi bài nộp này.

Bài làm tham khảo slide học phần, tài liệu hướng dẫn CNN và các phần liên quan trong sách *Deep Learning with Python*, ấn bản thứ hai của François Chollet. Kết cấu báo cáo tham khảo mẫu Assignment 03 được cung cấp; số liệu và hình ảnh thực nghiệm được tạo từ chương trình của Assignment 04, không lấy từ báo cáo mẫu.

Nội dung cần thực hiện	Mình chứng trong bài
Discover CNN	Lý thuyết, công thức, phép tính minh họa, kích thước và feature maps
Improved CNN	BatchNorm, residual, dropout; đối chiếu với baseline
From scratch / PyTorch / TensorFlow	Ba notebook thuật toán; mỗi notebook chạy hai mô hình trên ba dataset
Kết quả có thể kiểm tra	History, logits, nhãn, metric, confusion matrix và checkpoint
Bài nộp	Báo cáo PDF, notebook có đầu ra, README, mã nguồn GitHub

Bảng 1. Đối chiếu yêu cầu và thành phần bài nộp.

Kho mã nguồn: https://github.com/namnguyen05-vn/PTHTTM_Assignment-04. Các link Kaggle và hướng dẫn Anaconda có trong báo cáo và README. Bài thực hành có trợ lý AI hỗ trợ triển khai, kiểm tra và biên soạn; người nộp cần nắm được mã nguồn và các kết luận để trình bày, đồng thời tuân thủ quy định học phần.

MỤC LỤC

CHƯƠNG 1. BÀI TOÁN VÀ THIẾT KẾ THỰC NGHIỆM	3
1.3. Môi trường và các điều kiện so sánh	4
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT CNN	5
2.2. Convolution, pooling và số tham số	6
2.3. Loss, lan truyền ngược và Adam	7
2.4. BatchNorm, residual và dropout	8
2.5. Kiến trúc cụ thể dùng trong bài	9
CHƯƠNG 3. DỮ LIỆU VÀ TIỀN XỬ LÝ	10
3.3. Kiểm tra chất lượng và nguy cơ rò rỉ	11
3.4. Khảo sát MNIST	12
3.5. Khảo sát CIFAR10	13
3.6. Khảo sát CIFAR100	14
CHƯƠNG 4. CÀI ĐẶT VÀ KIỂM CHỨNG	15
4.2. Cài đặt bằng PyTorch	16
4.3. Cài đặt bằng TensorFlow/Keras	17
4.4. Kiểm tra gradient và đối chiếu giữa backend	18
CHƯƠNG 5. KẾT QUẢ THỰC NGHIỆM	19
5.2. Phân tích lỗi MNIST	20
5.3. Kết quả CIFAR10	21
5.4. Phân tích lỗi CIFAR10	22
5.5. Kết quả CIFAR100	23
5.6. Phân tích lỗi CIFAR100	24
5.7. Confusion matrix và recall từng lớp	25
5.8. So sánh tổng hợp và chi phí	26
5.9. Thử nghiệm ban đầu và điều chỉnh phần Dense	27
5.10. Giới hạn và độ tin cậy của kết quả	28
CHƯƠNG 6. KẾT LUẬN VÀ TÁI LẬP	29
6.3. Hướng dẫn chạy bằng Anaconda/Jupyter	30
6.4. Cấu trúc bài nộp và khả năng truy vết	31
TÀI LIỆU THAM KHẢO	32

CHƯƠNG 1. BÀI TOÁN VÀ THIẾT KẾ THỰC NGHIỆM

1.1. Mục tiêu và cách hiểu yêu cầu

Đầu vào là ảnh và đầu ra là một nhãn thuộc C lớp. Mạng trả về C logits; lớp có logit lớn nhất là nhãn dự đoán. MNIST là bài toán chữ số 10 lớp; CIFAR-10 là ảnh tự nhiên 10 lớp; CIFAR-100 dùng 100 nhãn fine, không gộp thành 20 nhãn coarse.

Đề không định nghĩa ngưỡng cho “2 big”. Bài này hiểu là hai dataset nhiều ảnh, phức tạp hơn MNIST, phù hợp khả năng huấn luyện từ đầu trên máy cá nhân. CIFAR-10 và CIFAR-100 đều có 60.000 ảnh màu, nhưng khác số lớp và mức phân biệt đối tượng. Cách chọn này đáp ứng hướng thực hành đã thống nhất, song chưa phải xác nhận của giảng viên rằng CIFAR được tính là “big”.

Tính trên cả train gốc và test, mỗi bộ CIFAR chứa 184,32 triệu giá trị kênh-pixel, so với 54,88 triệu ở MNIST, tương đương khoảng 3,36 lần. Vì vậy ảnh ít hơn về số mẫu nhưng lớn hơn về lượng giá trị đầu vào và khó hơn về nội dung phân loại.

1.2. Câu hỏi nghiên cứu

Thứ nhất, CNN viết bằng NumPy có gradient đúng và học được từ dữ liệu thực tế không? Thứ hai, cùng một kiến trúc có được biểu diễn tương đương trong NumPy, PyTorch và TensorFlow không? Thứ ba, việc thêm chuẩn hóa, residual và dropout làm thay đổi chất lượng phân loại ra sao ở ba mức độ khó?

Yếu tố	Thiết lập
Dữ liệu	Toàn bộ train/validation/test đã định nghĩa; không subsampling
Hai mô hình	Baseline và improved cuối cùng, khởi tạo mới cho mỗi lượt
Ba triển khai	NumPy trên CPU; PyTorch và TensorFlow trên GPU
Chọn mô hình	Epoch có validation cross-entropy thấp nhất
Đánh giá	Accuracy, macro precision/recall/F1, top-5, loss, lỗi từng lớp

Bảng 2. Thiết kế chung của 18 cấu hình.

Không dùng pretrained weights, transfer learning hoặc data augmentation trong thí nghiệm chính. Nhờ vậy, sự khác nhau giữa baseline và improved tập trung vào kiến trúc và regularization trong cùng ngân sách epoch; kết quả không được diễn giải là mức tốt nhất có thể đạt trên các benchmark này.

Một tham chiếu đơn giản là luôn đoán lớp xuất hiện nhiều nhất trong phần train (chọn chỉ số nhỏ nhất khi hòa): accuracy test khi đó là 11,35% ở MNIST, 10% ở CIFAR-10 và 1% ở CIFAR-100. Các CNN được so sánh với nhau trong bảng chính, đồng thời cần vượt xa tham chiếu này để chứng tỏ đã học thông tin từ ảnh.

1.3. Môi trường và các điều kiện so sánh

Thành phần	Môi trường thực nghiệm
Hệ điều hành	Windows; Jupyter/Anaconda
CPU / RAM	Intel Core i5-12500H / 16 GB
GPU	NVIDIA GeForce RTX 3050 Laptop, 4 GB VRAM
Python / NumPy	Python 3.10.20; NumPy 1.23.5
PyTorch	2.5.1+cu121
TensorFlow	2.10.1; CUDA 11.2; cuDNN 8.1
Kernel notebook	Python (Assignment 04)

Bảng 3. Máy và môi trường chạy thực tế.

Môi trường dùng Python 3.10 để tương thích TensorFlow 2.10.1. Theo hướng dẫn chính thức, 2.10 là phiên bản cuối hỗ trợ GPU trực tiếp trên Windows bản địa [12]. PyTorch dùng thư viện CUDA đi kèm gói cu121. Hai backend chạy trong tiến trình riêng để tránh xung đột DLL và trạng thái GPU.

Tập chia, chuẩn hóa pixel, batch size, thứ tự minibatch theo epoch và bộ trọng số khởi tạo được dùng chung. Trọng số NumPy được chuyển đúng thứ tự sang các framework; số tham số trainable được kiểm tra bằng assert. Các kernel GPU vẫn có sai khác số thực, dropout dùng luồng ngẫu nhiên riêng và Adam có chi tiết triển khai khác nhau.

Siêu tham số	Giá trị
Seed / batch size	42 / 128
Optimizer	Adam; learning rate 0,001; beta1 0,9; beta2 0,999; epsilon 1e-8
Epoch MNIST / CIFAR-10 / CIFAR-100	5 / 10 / 12
Chuẩn hóa	Pixel uint8 chuyển float32 và chia 255
Dropout improved	0,25; chỉ bật khi train
BatchNorm	Epsilon 1e-5; running = $0,9 \times cũ + 0,1 \times batch$

Bảng 4. Siêu tham số chung, không dò theo test.

Thời gian được đo trên máy đang đồng thời hoàn thiện các phần của bài. NumPy sử dụng CPU, hai framework sử dụng GPU; vì vậy số giây chỉ mô tả chi phí thực tế, không phải benchmark cô lập và không đủ kết luận framework nào nhanh hơn.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT CNN

2.1. Mạng học sâu như phép hợp thành hàm

Một mạng nơ-ron nhận tensor đầu vào và lần lượt áp dụng các hàm có tham số. Với CNN, chuỗi hàm thường gồm convolution, kích hoạt, pooling và lớp phân loại. Toàn bộ chuỗi tạo ra một hàm khả vi gần khắp nơi để tối ưu bằng gradient. Góc nhìn này kết nối ví dụ hợp thành hàm trong tài liệu hướng dẫn với mã forward/backward [1, 2].

$$z = f_L(f_{L-1}(\dots f_2(f_1(x; \theta_1); \theta_2) \dots); \theta_L)$$

Convolution khai thác tính cục bộ: một bộ lọc nhỏ nhìn vào vùng ảnh gần nhau để học cạnh, nét hoặc mẫu màu. Cùng bộ lọc được dùng ở nhiều vị trí, giảm số tham số so với kết nối đầy đủ đến từng vị trí độc lập. Các tầng sau kết hợp đặc trưng từ tầng trước thành biểu diễn phù hợp nhận cần dự đoán [3].

ReLU đặt các giá trị âm về 0. Nếu mọi tầng chỉ gồm các phép tuyến tính hoặc affine, nhiều tầng liên tiếp vẫn có thể gộp thành một phép affine; độ sâu khi đó không tạo ra khả năng biểu diễn phi tuyến cần thiết. Pooling giảm kích thước không gian, giúp phần sau gọn hơn nhưng cũng làm mất thông tin vị trí chi tiết.



Hình 1. Ảnh test MNIST và tám feature map sau Conv1 + ReLU của CNN NumPy baseline đã học. Mỗi bản đồ được tô màu theo thang riêng, dùng để xem mẫu đáp ứng chứ không so biên độ giữa kênh.

Các feature map thể hiện vùng mà từng bộ lọc đáp ứng. Không nên gán chắc chắn “bộ lọc này nhận biết chữ số” chỉ từ một ảnh minh họa. Tính hữu ích của đặc trưng cuối cùng được kiểm tra bằng chất lượng trên dữ liệu chưa dùng để cập nhật trọng số.

2.2. Convolution, pooling và số tham số

Trong mã và phần lớn framework, phép toán gọi là convolution thực hiện cross-correlation: kernel không được lật. Đầu ra ở một vị trí là tổng tích giữa kernel và cửa sổ đầu vào, cộng bias. NCHW lần lượt là số ảnh, số kênh, chiều cao và chiều rộng.

$$Y[n,o,i,j] = b[o] + \sum_{c,u,v} W[o,c,u,v] X[n,c,i+u-P,j+v-P]$$

$$H_{out} = \text{floor}((H + 2P - D(K-1) - 1)/S) + 1$$

Với kernel 3×3, padding 1, stride 1 và dilation 1, convolution giữ nguyên chiều rộng/cao. MaxPool 2×2, stride 2 đưa ảnh 28×28 về 14×14 rồi 7×7; ảnh 32×32 về 16×16 rồi 8×8. MaxPool không có tham số cần học.

Lớp	Công thức tham số	Ví dụ
Conv2D có bias	$(K^2 \times C_{in} + 1) \times C_{out}$	Conv 3→8, K=3: 224
Dense có bias	$(D_{in} + 1) \times D_{out}$	Dense 1.024→64: 65.600
BatchNorm	$2 \times \text{số kênh/đặc trưng}$	64 đặc trưng: 128 trainable
ReLU / MaxPool / Dropout	0	Không có trọng số học

Bảng 5. Quy tắc đếm tham số trainable.

Running mean và running variance của BatchNorm là trạng thái cập nhật theo dữ liệu, không phải tham số tối ưu bằng gradient nên không được tính vào cột trainable. Dense chiếm phần lớn tham số trong mạng nhỏ này do Flatten giữ mọi vị trí không gian.

Ví dụ số học độc lập

$$[2, 1, 3] \cdot [0,5; -1; 0,5] = 2 \times 0,5 - 1 + 3 \times 0,5 = 1,5$$

Notebook Discover thực thi ví dụ trên và assert kết quả. Việc kiểm tra phép tính nhỏ giúp phát hiện sai sót trong minh họa trước khi dùng công thức cho tensor lớn.

2.3. Loss, lan truyền ngược và Adam

Mạng xuất logits z . Softmax tạo xác suất p ; sparse cross-entropy dùng nhãn nguyên thay vì lưu one-hot. Để tránh tràn $\exp$, trừ logit lớn nhất trong từng mẫu trước khi tính softmax và $\log\text{-sum-exp}$.

$$p_{i,c} = \frac{\exp(z_{i,c} - \max_k z_{i,k})}{\sum_k \exp(z_{i,k} - \max_k z_{i,k})}$$

$$L = -\frac{1}{N} \sum_i \log p_{i,y_i}, \quad \frac{\partial L}{\partial z} = \frac{p - \text{onehot}(y)}{N}$$

Quy tắc dây chuyền cho phép truyền đạo hàm từ loss về từng lớp. Dense với $Y=XW+b$ có $dW=X^T G$, db là tổng G theo batch và $dX=GW^T$. ReLU giữ gradient tại đầu vào dương. MaxPool chuyển gradient đến vị trí cực đại đã lưu trong forward.

Với convolution, dW cộng đóng góp ở mọi vị trí và mọi mẫu. Gradient về đầu vào phải cộng dồn từ các cửa sổ chồng lấn. Bản NumPy lưu ma trận im2col ở forward, nhân ma trận ở backward và dùng vòng lặp theo vị trí kernel để col2im . Thiếu phép cộng dồn sẽ cho gradient sai ngay cả khi kích thước tensor vẫn hợp lệ.

Cập nhật Adam

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\hat{m}_t = m_t / (1 - \beta_1^t), \quad \hat{v}_t = v_t / (1 - \beta_2^t)$$

$$\theta_t = \theta_{t-1} - \alpha \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$$

Adam lưu trung bình động của gradient và bình phương gradient, đồng thời hiệu chỉnh lệch ban đầu [10]. Khởi tạo trọng số theo phương sai $2/\text{fan-in}$ phù hợp với các tầng ReLU [11]. Cài đặt NumPy thực hiện trực tiếp các biểu thức; các framework cung cấp optimizer nhưng vẫn dùng cùng giá trị siêu tham số.

Kiểm tra gradient bằng sai phân hữu hạn chỉ dùng tensor nhỏ và tránh vùng không trơn không cần thiết. Chênh lệch giữa đạo hàm giải tích và đạo hàm số được đối chiếu theo dung sai float32; bước kiểm tra này độc lập với việc mô hình đạt accuracy cao hay thấp.

2.4. BatchNorm, residual và dropout

Batch Normalization

BatchNorm chuẩn hóa theo thống kê minibatch rồi học hệ số scale γ và shift β [7]. Với ảnh, thống kê lấy theo N,H,W riêng từng kênh; với Dense, lấy theo batch riêng từng đặc trưng. Trong đánh giá, dùng running statistics đã tích lũy, không tính thống kê từ tập test.

$$\hat{x} = (x - \mu) / \sqrt{\sigma^2 + \epsilon}, \quad y = \gamma \hat{x} + \beta$$

$$dx = \frac{\gamma}{\sqrt{\sigma^2 + \epsilon}} [g - \text{mean}(g) - \hat{x} \text{mean}(g\hat{x})]$$

Ba triển khai dùng phương sai tổng thể. PyTorch mặc định có khác biệt trong cách cập nhật running variance; bài này viết lớp chuẩn hóa bằng tensor/autograd để khớp NumPy và Keras fused=False. Đây là lựa chọn phục vụ đối chiếu số học, cần ghi rõ khi so với nn.BatchNorm tiêu chuẩn.

Residual connection

$$y = \text{ReLU}(F(x) + x); F(x) = \text{BN}(\text{Conv}3 \times 3(x))$$

Nhánh identity giúp gradient truyền qua phép cộng, đồng thời nhánh F học phần biến đổi cần bổ sung [8]. Block dùng cùng 16 kênh và cùng kích thước không gian nên không cần projection. Backward phải cộng gradient từ nhánh identity với gradient đi qua Conv và BN.

Dropout

Trong train, dropout che ngẫu nhiên 25% đặc trưng ở phần phân loại; các phần tử giữ lại được chia cho 0,75 để bảo toàn kỳ vọng. Trong eval, dropout trở thành ánh xạ identity. Regularization có thể giảm phụ thuộc quá mức vào một số đặc trưng [9], nhưng cũng có thể làm học chậm khi mạng nhỏ hoặc số epoch ít.

Phiên bản cuối đặt BN sau Dense 64 trước ReLU, ngoài các BN ở convolution. Thử nghiệm ban đầu thiếu lớp chuẩn hóa này cho train/validation loss cao ở CIFAR-100. Việc bổ sung được kiểm tra bằng gradient và áp dụng thống nhất ở cả ba backend; phần kết quả giữ lại pilot để đối chiếu.

2.5. Kiến trúc cụ thể dùng trong bài

Khối	Baseline	Improved cuối
Trích xuất 1	Conv 3×3 , 8 $\rightarrow$ ReLU $\rightarrow$ Pool	Conv 3×3 , 8 $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Pool
Trích xuất 2	Conv 3×3 , 16 $\rightarrow$ ReLU	Conv 3×3 , 16 $\rightarrow$ BN $\rightarrow$ ReLU
Residual	Không có	ReLU(BN(Conv 16 $\rightarrow$ 16) + x)
Giảm kích thước	Pool $\rightarrow$ Flatten	Pool $\rightarrow$ Flatten
Phân loại	Dense 64 $\rightarrow$ ReLU $\rightarrow$ Dense C	Dense 64 $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Dropout 0,25 $\rightarrow$ Dense C

Bảng 6. Hai kiến trúc được triển khai tương đương trong ba backend.

Dataset	Sau Pool 1	Sau Pool 2	Flatten
MNIST	$8 \times 14 \times 14$	$16 \times 7 \times 7$	784
CIFAR-10/100	$8 \times 16 \times 16$	$16 \times 8 \times 8$	1.024

Bảng 7. Kích thước đặc trưng, không tính chiều batch.

Dataset	Baseline	Improved	Tăng tham số
MNIST	52.138	54.666	2.528
CIFAR10	67.642	70.170	2.528
CIFAR100	73.492	76.020	2.528

Bảng 8. Tham số trainable; dấu chấm trong số đếm phân tách hàng nghìn.

Baseline có 2 convolution và 2 dense; improved có 3 convolution và 2 dense, thêm 4 BatchNorm. Số “lớp” có thể khác nếu tính cả activation và pooling, vì vậy báo cáo mô tả từng khối thay vì chỉ dùng một nhãn như CNN 3 lớp hay 5 lớp.

Các cải tiến được đánh giá như một gói kiến trúc. Chênh lệch baseline/improved không đủ chứng minh đóng góp nhân quả riêng của residual, BN hoặc dropout. Pilot bổ sung một đối chiếu về BN ở Dense nhưng chưa phải ablation đầy đủ của mọi thành phần.

CHƯƠNG 3. DỮ LIỆU VÀ TIỀN XỬ LÝ

3.1. Nguồn Kaggle và quy mô

Dataset	Train gốc	Test	Lớp / định dạng
MNIST	60.000	10.000	10; ảnh xám 28×28
CIFAR-10	50.000	10.000	10; RGB 32×32
CIFAR-100	50.000	10.000	100 fine; RGB 32×32

Bảng 9. Quy mô gốc kiểm tra trên các file tải về.

[4] MNIST: <https://www.kaggle.com/datasets/oddrational/mnist-in-csv>.

[5] CIFAR10: <https://www.kaggle.com/datasets/pankrzysiu/cifar10-python>.

[6] CIFAR100: <https://www.kaggle.com/datasets/fedesoriano/cifar100>.

CIFAR-10 và CIFAR-100 là các bộ ảnh của Krizhevsky, Nair và Hinton [13]. MNIST dùng bản CSV có cột nhãn và 784 cột pixel. CIFAR dùng bản Python, chứa mảng ảnh, nhãn và metadata; chương trình chỉ cho phép các đối tượng NumPy cần thiết khi đọc định dạng pickle.

3.2. Quy trình nạp dữ liệu

Script tải ZIP công khai từ Kaggle, kiểm tra đường dẫn giải nén không vượt thư mục đích, lưu SHA-256 và danh sách file. Mảng ảnh được chuyển về NCHW, nhãn int64, tên lớp dạng chuỗi. File NPZ chuẩn hóa cấu trúc giúp cả ba backend dùng cùng dữ liệu đầu vào.

Tập validation được tách bằng shuffle theo lớp với seed 42. 10% mỗi lớp được làm tròn riêng; vì vậy MNIST có 5.999 mẫu validation. Không chuyển ảnh giữa train và test chính thức. Tất cả chỉ số split được xuất ra file để đối chiếu, thay vì chỉ ghi seed và chia lại tùy framework.

Dataset	Train	Validation	Test
MNIST	54,001	5,999	10,000
CIFAR10	45,000	5,000	10,000
CIFAR100	45,000	5,000	10,000

Bảng 10. Phân chia thực tế dùng trong 18 thí nghiệm.

3.3. Kiểm tra chất lượng và nguy cơ rò rỉ

Các ảnh có kiểu uint8 và pixel trong $[0, 255]$. Nhân nằm trong miền lớp hợp lệ. Train và validation không giao nhau theo chỉ số; tổng số chỉ số bằng toàn bộ tập training gốc. MNIST và CIFAR-10 không có ảnh pixel trùng hoàn toàn giữa các phần theo phép kiểm tra SHA-256 đã thực hiện.

Dataset	Ảnh train gốc duy nhất	Hash chung train-val	Hash chung train-test	Hash chung val-test
MNIST	60000	0	0	0
CIFAR10	50000	0	0	0
CIFAR100	49986	4	8	2

Bảng 11. Kiểm tra trùng hoàn toàn theo nội dung pixel; số liệu là số hash chung.

CIFAR-100 có 49.986 ảnh pixel duy nhất trong 50.000 mẫu training gốc. Có một số nội dung ảnh xuất hiện ở hai phần khác nhau mặc dù chỉ số mẫu khác nhau. Điều này làm suy yếu giả thiết độc lập hoàn toàn của các phần dữ liệu; riêng các ảnh trùng train-validation có thể ảnh hưởng nhẹ việc chọn checkpoint.

Bài giữ nguyên benchmark và công khai hạn chế, không thay đổi split hoặc lọc mẫu sau khi xem mô hình dự đoán đúng/sai. Phần phân tích độ nhạy tính thêm accuracy trên các ảnh test không trùng training gốc; đây là kiểm tra bổ sung, không thay thế chỉ số benchmark chính thức.

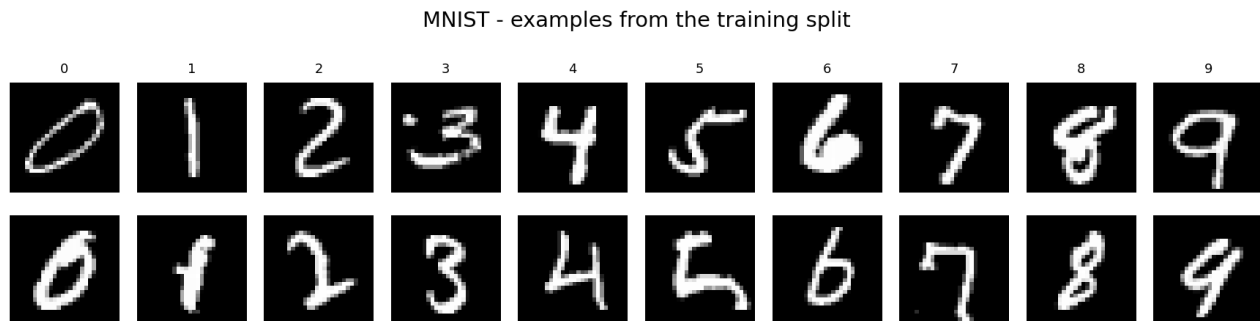
Exact hash chỉ phát hiện ảnh có pixel giống hệt, không phát hiện các bản cắt, đổi màu hoặc ảnh gần giống. Do đó kết quả kiểm tra bằng 0 không đồng nghĩa không còn mọi dạng trùng lặp ngữ nghĩa. Nghiên cứu tiếp theo nên xây dựng split theo nhóm nội dung từ đầu và đánh giá trên một holdout mới.

Kiểm soát tiền xử lý

Chia 255 là biến đổi cố định, không dùng thống kê test. Không chuẩn hóa theo mean/std của toàn bộ dữ liệu, không tăng cường dữ liệu trước khi chia tập. BatchNorm ở test dùng thống kê đã học, dropout tắt. Những lựa chọn này tránh việc bước chuẩn bị hoặc chế độ mô hình vô tình dùng thông tin test để huấn luyện.

3.4. Khảo sát MNIST

MNIST gồm chữ số viết tay từ 0 đến 9 trên ảnh xám 28×28 . Nền đơn giản và đối tượng được căn tương đối đồng nhất giúp mạng nhỏ học nhanh. Tuy vậy, kiểu chữ khác nhau, nét mờ hoặc chữ số có hình dạng gần nhau vẫn gây lỗi.



Hình 2. Hai ảnh mỗi lớp minh họa lấy từ phần train.

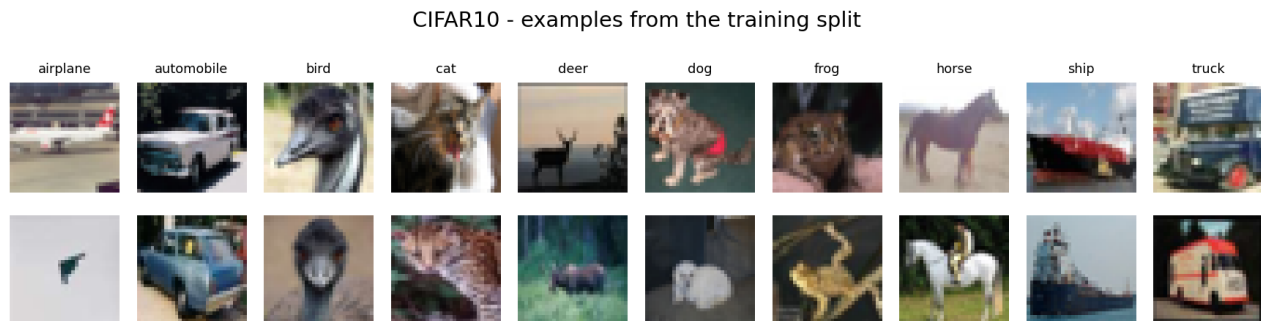


Hình 3. Phân bố số mẫu trong phần train sau chia validation; chỉ số lớp giữ theo metadata dữ liệu.

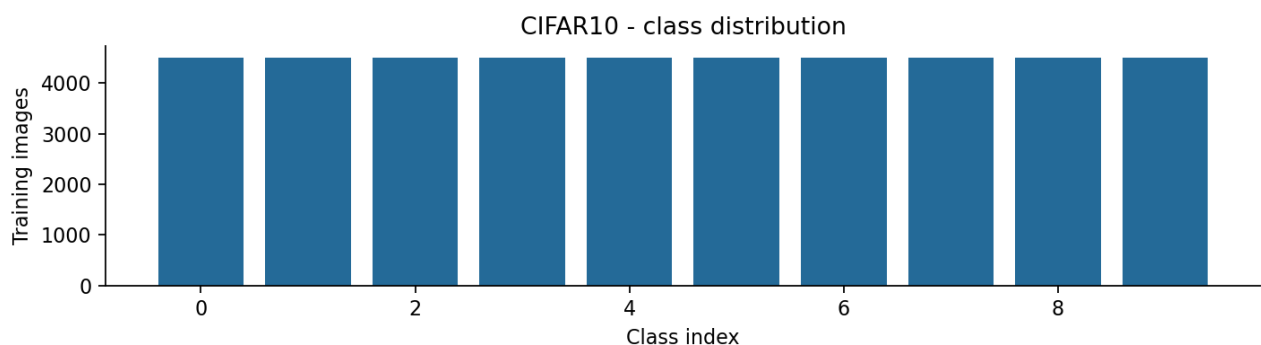
Số mẫu mỗi chữ số khác nhau nhưng không quá chênh lệch. Lớp phổ biến nhất trong phần train là chữ số 1; luôn dự đoán 1 chỉ đạt 11,35% test accuracy. Macro-F1 giúp cho các lớp trọng số ngang nhau khi tổng hợp, bổ sung cho accuracy chịu ảnh hưởng số mẫu. Toàn bộ ảnh giữ nguyên 28×28 , không cần phóng to để khớp CIFAR.

3.5. Khảo sát CIFAR10

CIFAR-10 có 10 lớp: airplane, automobile, bird, cat, deer, dog, frog, horse, ship và truck. Ảnh RGB kích thước nhỏ chứa thay đổi về nền, tư thế, ánh sáng và tỷ lệ đối tượng. So với MNIST, mạng phải phân biệt cấu trúc tự nhiên thay vì chủ yếu hình dạng nét trên nền đơn giản.



Hình 4. Hai ảnh mỗi lớp minh họa lấy từ phần train.



Hình 5. Phân bố số mẫu trong phần train sau chia validation; chỉ số lớp giữ theo metadata dữ liệu.

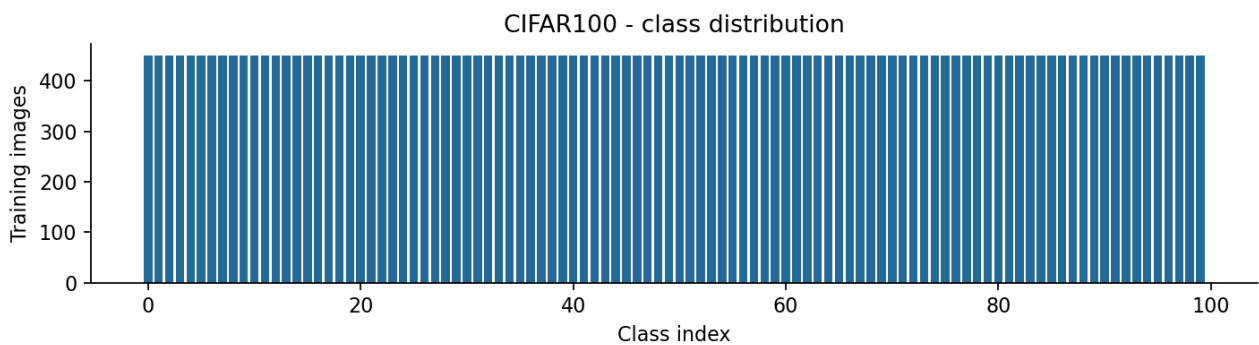
Sau chia, mỗi lớp có 4.500 ảnh train, 500 validation và 1.000 test. Vì test cân bằng, macro-recall bằng accuracy; macro-F1 vẫn cung cấp thông tin khác do chịu ảnh hưởng precision của từng lớp. Không cần class weighting để bù mất cân bằng ở benchmark này. Dữ liệu có cấu trúc không gian rõ ràng, phù hợp Conv2D hơn ba bộ dạng bảng/văn bản ban đầu.

3.6. Khảo sát CIFAR100

CIFAR-100 tăng bài toán lên 100 lớp fine với 500 ảnh training gốc và 100 ảnh test mỗi lớp. Sau chia validation, mỗi lớp còn 450 ảnh để cập nhật trọng số. Nhiều lớp thuộc cùng nhóm ngữ nghĩa hoặc có màu/kết cấu gần nhau; độ phân giải 32×32 làm mất các dấu hiệu phân biệt nhỏ.



Hình 6. Hai ảnh mỗi lớp minh họa lấy từ phần train. Với CIFAR-100, chọn 10 lớp có chỉ số cách nhau 10, không đại diện toàn bộ 100 lớp.



Hình 7. Phân bố số mẫu trong phần train sau chia validation; chỉ số lớp giữ theo metadata dữ liệu.

Mỗi lớp có 450 ảnh train, 50 validation và 100 test. Số mẫu trên mỗi lớp chỉ bằng một phần mười CIFAR-10 dù số ảnh toàn bộ bằng nhau. Vì vậy cần kiểm tra cả macro-F1, top-5 và các cặp lớp hay nhầm. Các số lượng và hình minh họa ở đây được tạo trực tiếp từ file tải về; hạn chế trùng pixel được giữ trong phần kiểm toán dữ liệu.

CHƯƠNG 4. CÀI ĐẶT VÀ KIỂM CHỨNG

4.1. CNN từ đầu bằng NumPy

Module `numpy_cnn.py` chỉ dùng NumPy cho phép tính mạng và cập nhật. Mỗi lớp cung cấp `forward`, `backward`, danh sách tham số/gradient và state. CNN gọi các lớp theo thứ tự ở `forward` và đảo thứ tự ở `backward`. Thiết kế này thể hiện trực tiếp quy tắc hợp thành hàm và giúp kiểm tra từng lớp độc lập.

Thành phần	Cách triển khai
Conv2D	<code>sliding_window_view</code> → <code>im2col</code> → nhân ma trận; backward tính <code>dW</code> , <code>db</code> và <code>col2im</code>
ReLU / MaxPool	Lưu mask hoặc <code>argmax</code> ở train; gradient chỉ về vị trí đã chọn
BatchNorm 2D/1D	Tự tính mean/variance và đạo hàm; cùng state running cho eval
Residual	Lưu nhánh Conv+BN; cộng gradient nhánh identity
Dense / Dropout	Nhân ma trận; inverted dropout dùng RNG
Loss / optimizer	Stable softmax cross-entropy và Adam có bias correction

Bảng 12. Các khối thuật toán từ đầu.

Lớp convolution dùng nhân ma trận của NumPy để tăng tốc, nhưng công thức `forward/backward` do chương trình tự tổ chức. Việc sử dụng BLAS phía dưới NumPy không tương đương dùng `autograd`: chương trình vẫn phải tạo `dW`, `db`, `dX` và cập nhật đúng cho từng tham số.

Ở train, lớp lưu các giá trị cần cho backward; ở eval, không tạo cache không cần thiết. Checkpoint NPZ lưu trọng số và running statistics. Sau khi huấn luyện, chương trình khôi phục checkpoint tốt nhất theo validation rồi mới tính test logits.

Bài toán thử nhỏ và kiểm tra sai phân hữu hạn giúp phát hiện lỗi cài đặt trước các lượt huấn luyện đầy đủ. Đạt loss giảm trên dữ liệu thật là bằng chứng mô hình học được, nhưng không thay thế kiểm chứng đạo hàm vì một số lỗi gradient vẫn có thể cho đường loss giảm.

4.2. Cài đặt bằng PyTorch

TorchCNN kế thừa `nn.Module`, biểu diễn các khối bằng `nn.Sequential`, `nn.Conv2d`, `nn.Linear`, `activation`, `pooling` và `dropout`. Residual được viết thành module riêng. PyTorch tự theo dõi đồ thị phép toán và tính gradient khi gọi `loss.backward()`.

`zero_grad` → `forward` → `cross_entropy` → `backward` → `optimizer.step`

Trước mỗi minibatch, xóa gradient cũ bằng `zero_grad(set_to_none=True)` để tránh cộng gradient ngoài ý muốn. Trong đánh giá, gọi `eval` và `no_grad` để tắt dropout, dùng running statistics và không giữ đồ thị gradient. Dữ liệu batch được chuyển từ NumPy sang device CUDA.

Trọng số convolution của PyTorch và NumPy đều dùng thứ tự Cout,Cin,K,K. Dense của NumPy có Din,Dout, trong khi `nn.Linear` lưu Dout,Din nên phải chuyển vị. Nếu quên chuyển vị hoặc đổi thứ tự Flatten, chương trình có thể vẫn chạy nhưng không còn biểu diễn cùng mạng.

PopulationBatchNorm viết bằng phép toán tensor và autograd để thống nhất cách cập nhật phương sai chạy. Đây không phải yêu cầu thông thường của mọi mô hình PyTorch; nó phục vụ thí nghiệm cần đổi chiều số học giữa ba triển khai. Các kiểm tra train/eval xác nhận hành vi của lớp này so với NumPy.

Nhiệm vụ	Cơ chế
Chế độ train	<code>model.train()</code> ; tính gradient và cập nhật Adam
Chế độ test	<code>model.eval()</code> ; <code>torch.no_grad()</code>
Checkpoint	<code>state_dict</code> lưu bằng <code>torch.save</code> ; tải <code>weights_only=True</code>
Thiết lập GPU	Tắt TF32; cuDNN deterministic; không benchmark tự chọn thuật toán
Giới hạn tái lập	Một seed không bảo đảm mọi máy/kernel cho cùng từng bit

Bảng 13. Các điểm cài đặt PyTorch liên quan đến tính đúng đắn.

4.3. Cài đặt bằng TensorFlow/Keras

Mô hình TensorFlow dùng Functional API. Input có thứ tự NHWC theo thói quen Keras. Convolution và pooling giữ cùng hình học với hai triển khai còn lại. Trước Flatten, Permute chuyển NHWC về NCHW để Dense nhìn cùng thứ tự đặc trưng.

Vòng lặp tùy chỉnh dùng GradientTape để lấy đạo hàm của sparse softmax cross-entropy theo trainable_weights. tf.function biên dịch bước train và infer nhằm giảm chi phí gọi Python. Vì epoch đầu có thêm thời gian biên dịch/khởi tạo kernel, không nên coi thời gian của nó như các epoch sau.

GradientTape: logits $\rightarrow$ loss $\rightarrow$ gradient(loss, weights) $\rightarrow$ apply_gradients

Keras BatchNormalization dùng momentum 0,9 và fused=False, epsilon 1e-5. Giá trị momentum ở Keras là trọng số của thống kê cũ, trong khi cách diễn đạt thường dùng ở NumPy/PyTorch có thể là trọng số của batch mới. Bài ghi rõ công thức running = $0,9 \times \text{cũ} + 0,1 \times \text{batch}$ để tránh hiểu nhầm.

Checkpoint H5 lưu weights và running statistics; sau chọn epoch, load_weights khôi phục mô hình để đánh giá. Optimizer Adam dùng learning rate 0,001 và epsilon 1e-8, nhưng vị trí epsilon trong công thức hiệu chỉnh có thể khác bản NumPy/PyTorch. Cộng với sai số GPU và dropout, điều này giải thích vì sao cùng khởi tạo chưa đủ tạo đường học giống hệt.

Vấn đề	Cách xử lý
Khác layout	NCHW $\rightarrow$ NHWC khi nhập; Permute trước Flatten
GPU Windows	TensorFlow 2.10.1, CUDA 11.2, cuDNN 8.1
Bộ nhớ GPU	Memory growth được cấu hình trước tạo model
Tách framework	Huấn luyện trong subprocess riêng
Trọng số ban đầu	Conv chuyển OIHW $\rightarrow$ HWIO; Dense giữ Din,Dout

Bảng 14. Các chi tiết bảo đảm TensorFlow tương ứng kiến trúc gốc.

4.4. Kiểm tra gradient và đối chiếu giữa backend

Bộ kiểm tra NumPy gồm 9 test: gradient convolution, dense, BatchNorm ảnh, BatchNorm sau Dense, max pooling, residual; ổn định loss; shape/số tham số; khả năng học một bài toán nhỏ. Tất cả đã chạy thành công. Sai phân hữu hạn kiểm tra các phần tử được chọn với bước 0,002 và dung sai phù hợp float32.

Backend so với NumPy	Số cấu hình	Sai số logits lớn nhất	Sai số gradient Conv1 lớn nhất
pytorch	6	5.01e-06	8.64e-06
tensorflow	6	5.01e-06	1.59e-05

Bảng 15. Đối chiếu trên CPU với dropout tắt; cả 12 cấu hình đều đạt dung sai.

Mỗi cấu hình so sánh logits khi train, logits khi eval sau cập nhật running statistics, gradient đầu vào và gradient trọng số Conv1. Kiểm tra dùng batch nhỏ chung, trọng số ban đầu chung, và tắt dropout để bỏ khác biệt ngẫu nhiên. Gradient Conv1 ở cuối chuỗi lan truyền đi qua phần lớn cấu trúc mạng, nên hữu ích hơn chỉ so shape.

Không kiểm tra bitwise equality. Dung sai assert_allclose là atol 2e-4 và rtol 3e-4; sai số quan sát nhỏ hơn nhiều so với giới hạn này. Đối chiếu này xác nhận phép tính cài đặt tương ứng trong các trường hợp thử, không khẳng định mọi hành vi ở mọi đầu vào đã được chứng minh tuyệt đối.

Kiểm chứng đầu ra sau huấn luyện

Script phân tích đọc lại logits và nhãn của cả 18 lượt, tính lại metric, đối chiếu confusion matrix CSV và kiểm tra best_epoch đúng vị trí validation loss nhỏ nhất. Mỗi history có đủ số epoch đã cấu hình. Các notebook được thực thi và kiểm tra không có ô lỗi trước khi đóng gói.

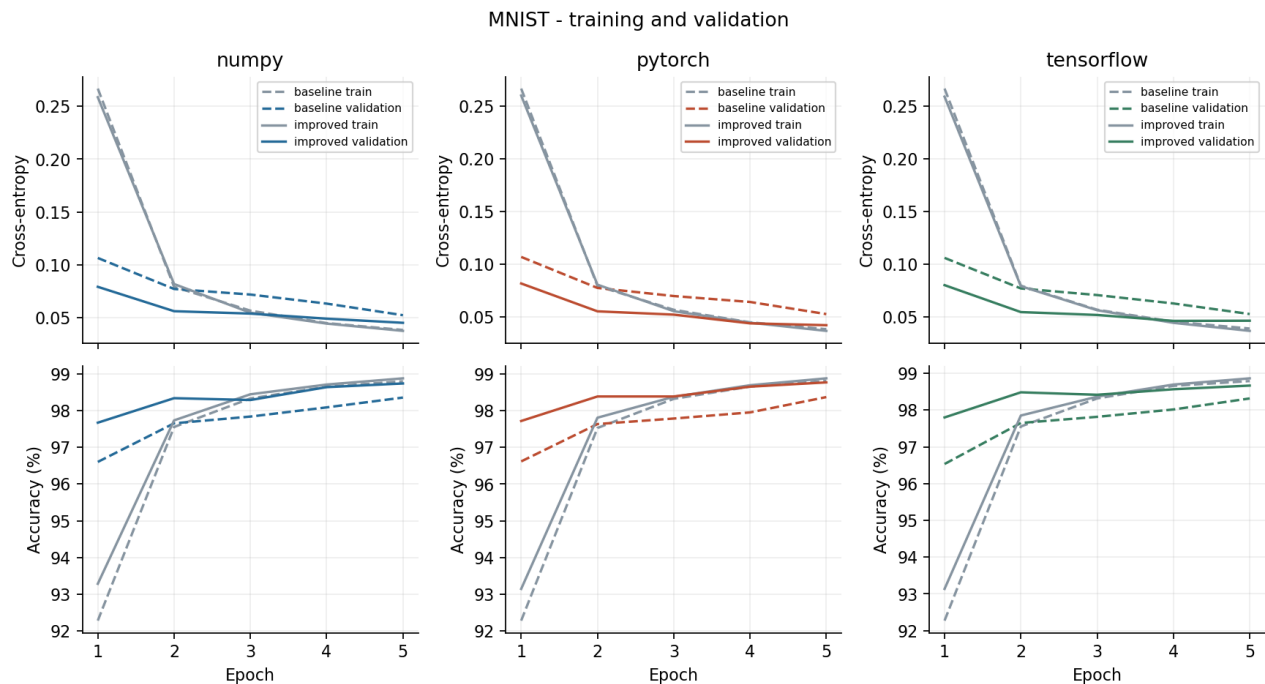
File verification JSON, history CSV và checkpoint đi kèm repository giúp người đọc truy vết các khẳng định thay vì chỉ xem bảng đã làm tròn trong PDF.

CHƯƠNG 5. KẾT QUẢ THỰC NGHIỆM

5.1. Kết quả MNIST

Backend	Mô hình	Acc. (%)	Macro-F1	Top-5 (%)	Epoch tốt
numpy	baseline	98,69	0,9868	99,98	5
numpy	improved	98,92	0,9891	99,99	5
pytorch	baseline	98,63	0,9862	99,97	5
pytorch	improved	99,03	0,9902	99,99	5
tensorflow	baseline	98,68	0,9867	99,97	5
tensorflow	improved	98,75	0,9875	100,00	4

Bảng 16. Kết quả test MNIST trên 10.000 ảnh; epoch chọn bằng validation loss.

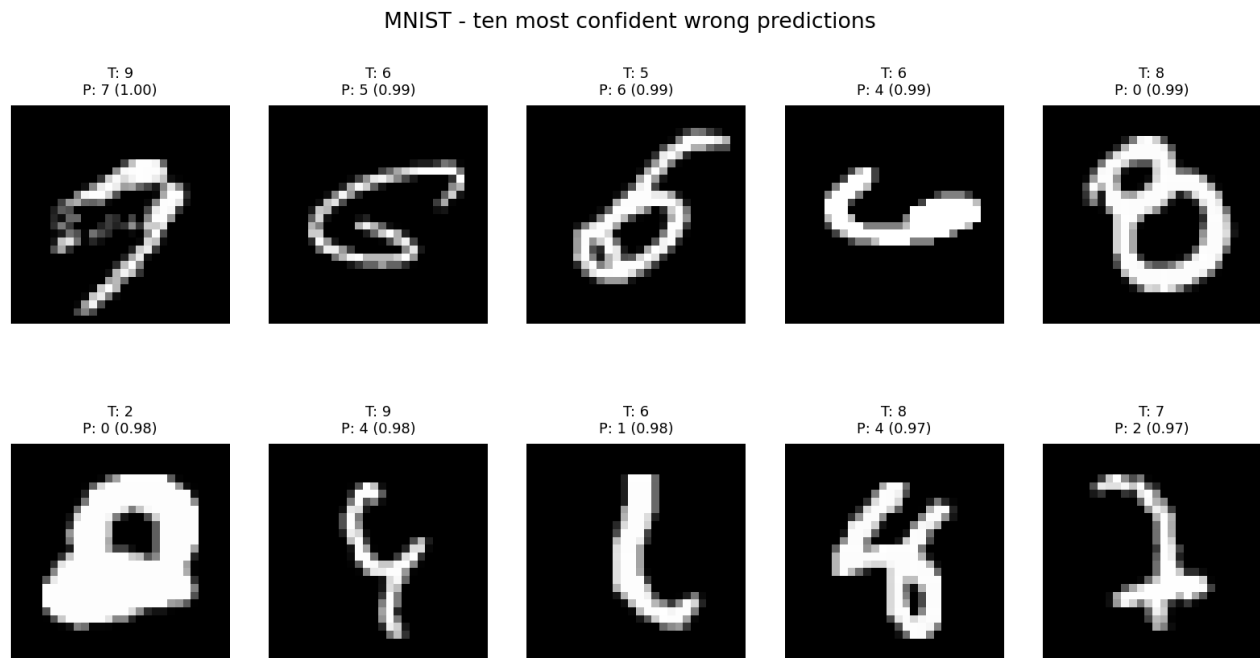


Hình 8. Loss và accuracy train/validation của sáu cấu hình MNIST. Nét đứt: baseline; nét liền: improved; xám: train; màu: validation.

Mức thay đổi accuracy khi dùng improved lần lượt ở NumPy, PyTorch, TensorFlow là +0,23, +0,40, +0,07 điểm phần trăm. Đây là các lượt chạy riêng seed 42; không phải trung bình nhiều seed. Chọn theo validation loss có thể lấy epoch khác epoch cuối hoặc epoch có validation accuracy cao nhất.

5.2. Phân tích lỗi MNIST

Mô hình minh họa là pytorch improved, được chọn vì có best validation loss thấp nhất trong sáu lượt của dataset. Các ảnh bên dưới là dự đoán sai tự tin nhất; không dùng chúng để thay đổi mô hình sau khi hoàn tất thí nghiệm chính.



Hình 9. Mười ảnh dự đoán sai có confidence cao nhất. T là nhãn thật, P là nhãn dự đoán; confidence là softmax lớn nhất, chưa được hiệu chỉnh xác suất.

Nhãn thật	Bị nhầm thành	Số ảnh
9	4	8
4	9	7
9	7	6
2	7	6
9	0	5

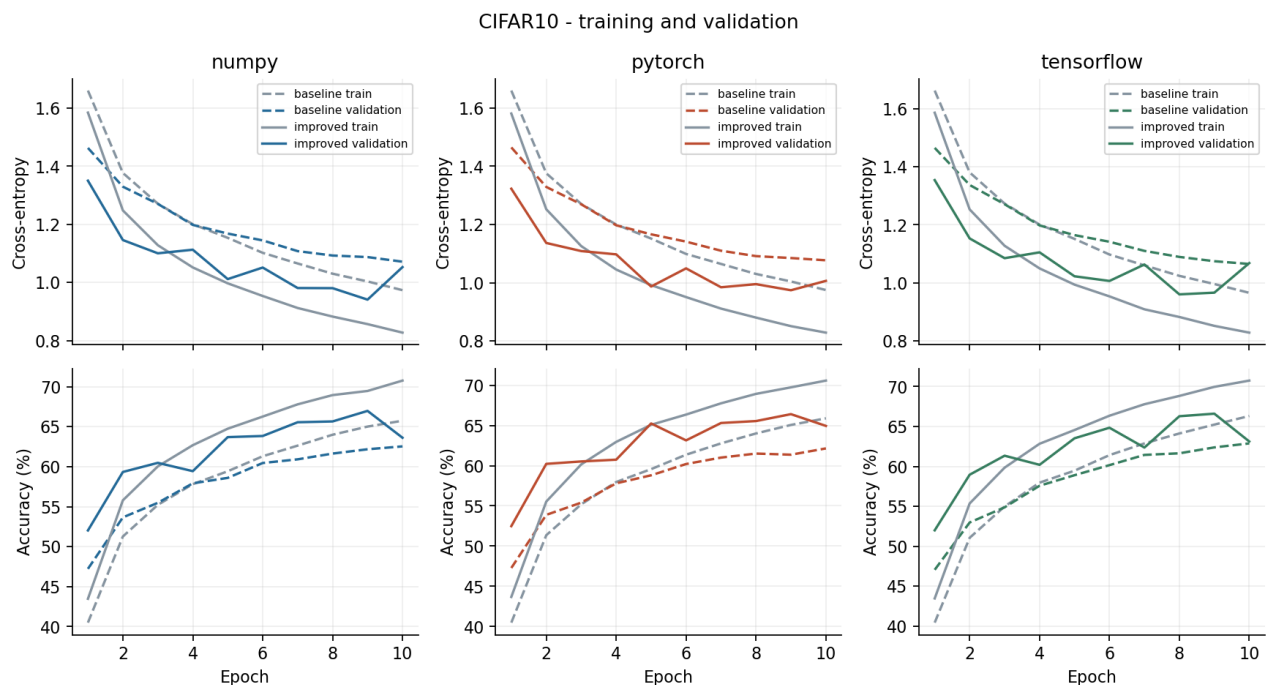
Bảng 17. Năm hướng nhầm lẫn nhiều nhất trong mô hình minh họa.

Các lỗi còn lại thường xuất hiện ở hình dạng nét khó phân biệt hoặc cách viết khác thông thường. Confidence cao ở một dự đoán sai cho thấy softmax không đồng nghĩa độ chắc chắn đã hiệu chỉnh. Chỉ từ các ảnh này chưa thể kết luận nhãn dataset sai; cần xem lại dữ liệu gốc và tiêu chí gán nhãn trước khi chỉnh nhãn.

5.3. Kết quả CIFAR10

Backend	Mô hình	Acc. (%)	Macro-F1	Top-5 (%)	Epoch tốt
numpy	baseline	63,15	0,6286	96,30	10
numpy	improved	66,43	0,6634	96,83	9
pytorch	baseline	63,03	0,6266	96,24	10
pytorch	improved	65,26	0,6531	96,70	9
tensorflow	baseline	63,01	0,6264	96,36	10
tensorflow	improved	66,02	0,6588	96,83	8

Bảng 18. Kết quả test CIFAR10 trên 10.000 ảnh; epoch chọn bằng validation loss.



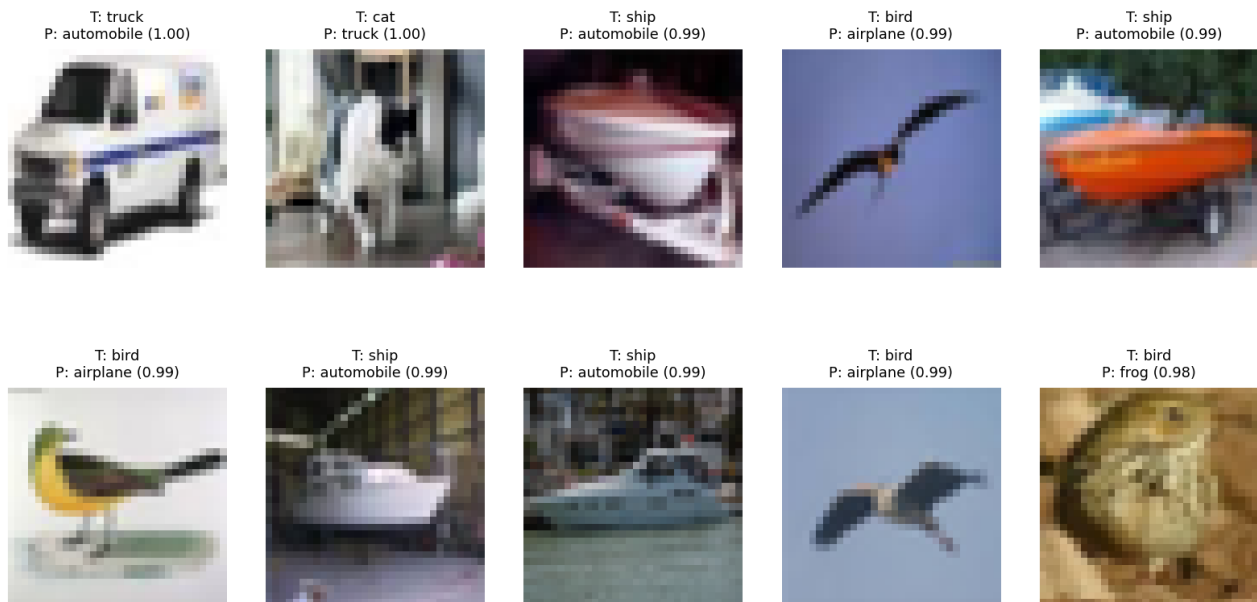
Hình 10. Loss và accuracy train/validation của sáu cấu hình CIFAR10. Nét đứt: baseline; nét liền: improved; xám: train; màu: validation.

Mức thay đổi accuracy khi dùng improved lần lượt ở NumPy, PyTorch, TensorFlow là +3,28, +2,23, +3,01 điểm phần trăm. Đây là các lượt chạy riêng seed 42; không phải trung bình nhiều seed. Chọn theo validation loss có thể lấy epoch khác epoch cuối hoặc epoch có validation accuracy cao nhất.

5.4. Phân tích lỗi CIFAR10

Mô hình minh họa là numpy improved, được chọn vì có best validation loss thấp nhất trong sáu lượt của dataset. Các ảnh bên dưới là dự đoán sai tự tin nhất; không dùng chúng để thay đổi mô hình sau khi hoàn tất thí nghiệm chính.

CIFAR10 - ten most confident wrong predictions



Hình 11. Mười ảnh dự đoán sai có confidence cao nhất. T là nhãn thật, P là nhãn dự đoán; confidence là softmax lớn nhất, chưa được hiệu chỉnh xác suất.

Nhãn thật	Bị nhầm thành	Số ảnh
cat	dog	193
dog	cat	154
deer	bird	117
ship	airplane	114
truck	automobile	113

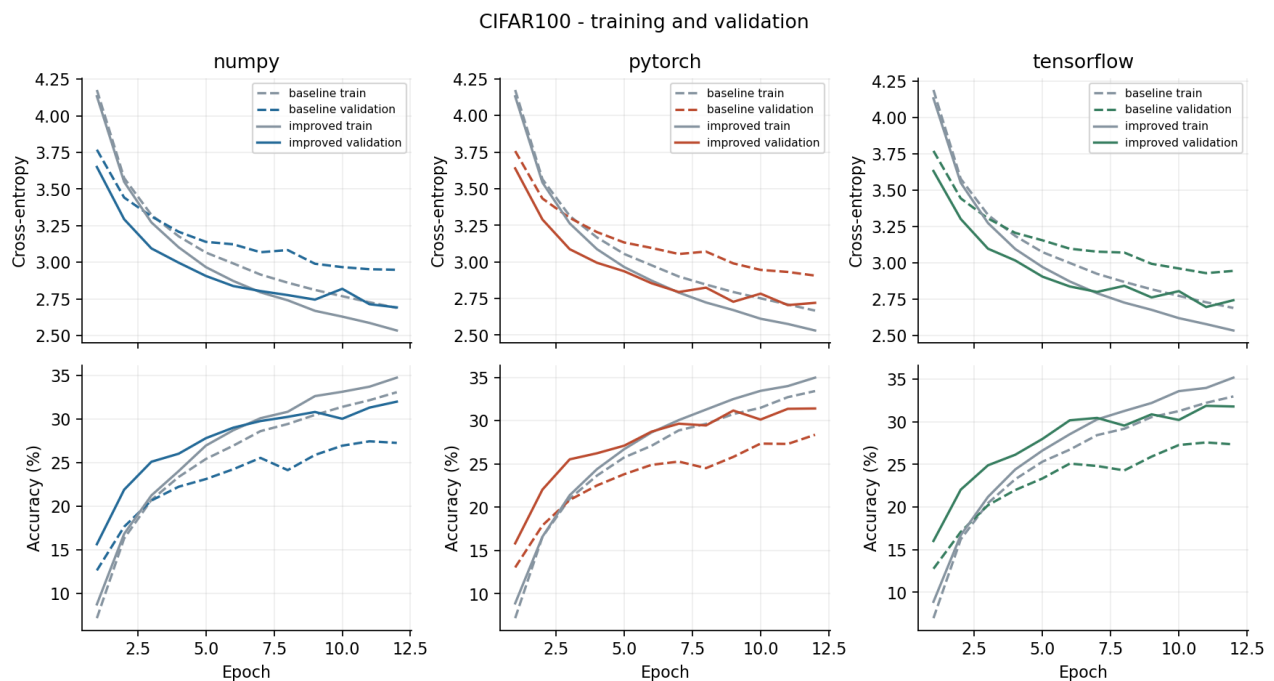
Bảng 19. Năm hướng nhầm lẫn nhiều nhất trong mô hình minh họa.

Các nhóm động vật hoặc phương tiện có thể chia sẻ hình dạng, màu và nền. CNN nhỏ nhìn ảnh 32×32 nên chi tiết phân biệt bị hạn chế. Cần đổi chiều tần suất nhầm với toàn bộ số mẫu mỗi lớp; một số ví dụ nổi bật không đủ chứng minh mô hình luôn dựa vào nền ảnh.

5.5. Kết quả CIFAR100

Backend	Mô hình	Acc. (%)	Macro-F1	Top-5 (%)	Epoch tốt
numpy	baseline	28,23	0,2735	57,95	12
numpy	improved	32,42	0,3168	63,02	12
pytorch	baseline	28,84	0,2773	58,88	12
pytorch	improved	32,15	0,3098	62,35	11
tensorflow	baseline	28,54	0,2744	58,19	11
tensorflow	improved	32,54	0,3127	62,85	11

Bảng 20. Kết quả test CIFAR100 trên 10.000 ảnh; epoch chọn bằng validation loss.



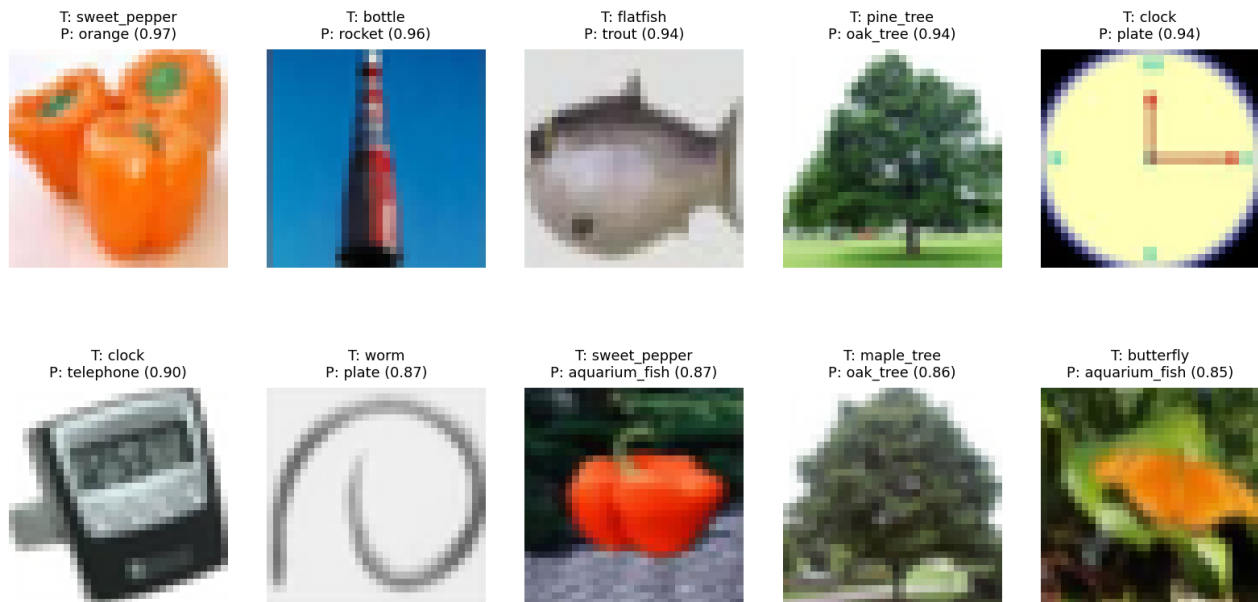
Hình 12. Loss và accuracy train/validation của sáu cấu hình CIFAR100. Nét đứt: baseline; nét liền: improved; xám: train; màu: validation.

Mức thay đổi accuracy khi dùng improved lần lượt ở NumPy, PyTorch, TensorFlow là +4,19, +3,31, +4,00 điểm phần trăm. Đây là các lượt chạy riêng seed 42; không phải trung bình nhiều seed. Chọn theo validation loss có thể lấy epoch khác epoch cuối hoặc epoch có validation accuracy cao nhất.

5.6. Phân tích lỗi CIFAR100

Mô hình minh họa là numpy improved, được chọn vì có best validation loss thấp nhất trong sáu lượt của dataset. Các ảnh bên dưới là dự đoán sai tự tin nhất; không dùng chúng để thay đổi mô hình sau khi hoàn tất thí nghiệm chính.

CIFAR100 - ten most confident wrong predictions



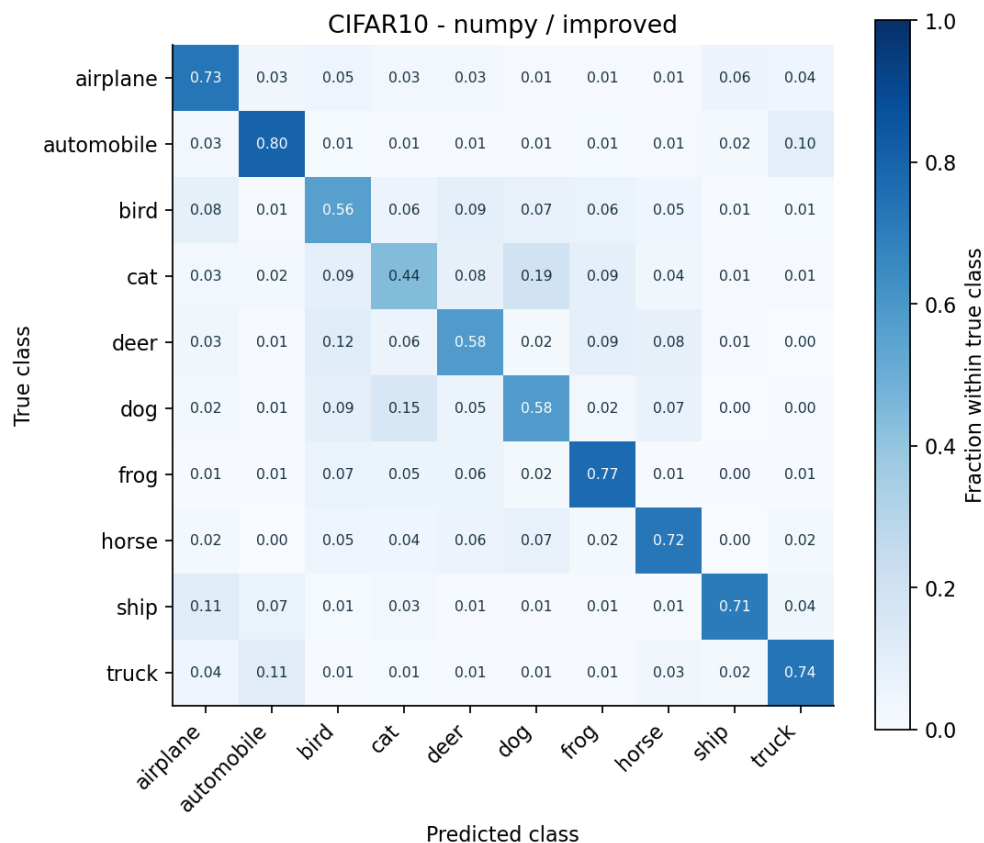
Hình 13. Mười ảnh dự đoán sai có confidence cao nhất. T là nhãn thật, P là nhãn dự đoán; confidence là softmax lớn nhất, chưa được hiệu chỉnh xác suất.

Nhãn thật	Bị nhầm thành	Số ảnh
ray	shark	25
maple_tree	oak_tree	23
whale	dolphin	20
plate	clock	19
willow_tree	oak_tree	18

Bảng 21. Năm hướng nhầm lẫn nhiều nhất trong mô hình minh họa.

Bài toán 100 lớp đòi hỏi phân biệt tinh hơn trong cùng độ phân giải và cùng phần trích xuất nhỏ. Macro-F1 thấp hơn accuracy ở một số lượt cho thấy chất lượng không đồng đều giữa lớp và phân phối dự đoán. Top-5 bổ sung góc nhìn về việc nhãn thật còn nằm trong nhóm ứng viên mạnh hay đã bị loại xa.

5.7. Confusion matrix và recall từng lớp

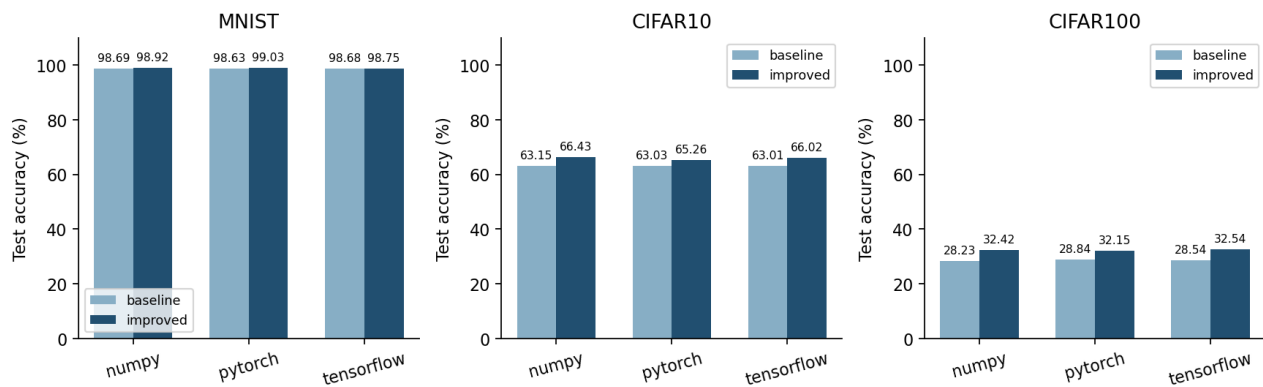


Hình 14. Confusion matrix CIFAR-10 chuẩn hóa theo nhãn thật. Ô đường chéo là recall của từng lớp; màu đậm ngoài đường chéo biểu thị nhầm lẫn có hệ thống.

Ma trận lưu đầy đủ dưới dạng CSV cho từng lượt, bao gồm CIFAR-100 kích thước 100×100 . Hàng là nhãn thật, cột là nhãn dự đoán. Khi chuẩn hóa theo hàng, tổng mỗi hàng bằng 1; nhờ vậy có thể đọc tỷ lệ nhầm thay vì chỉ số lượng.

Notebook so sánh kèm hình ma trận MNIST/CIFAR-100 và biểu đồ recall của các lớp thấp nhất. Không thu nhỏ toàn bộ 100 nhãn vào một bảng chữ khó đọc trong báo cáo; dữ liệu chi tiết vẫn có thể xem trong file đầu ra và hình ở repository.

5.8. So sánh tổng hợp và chi phí



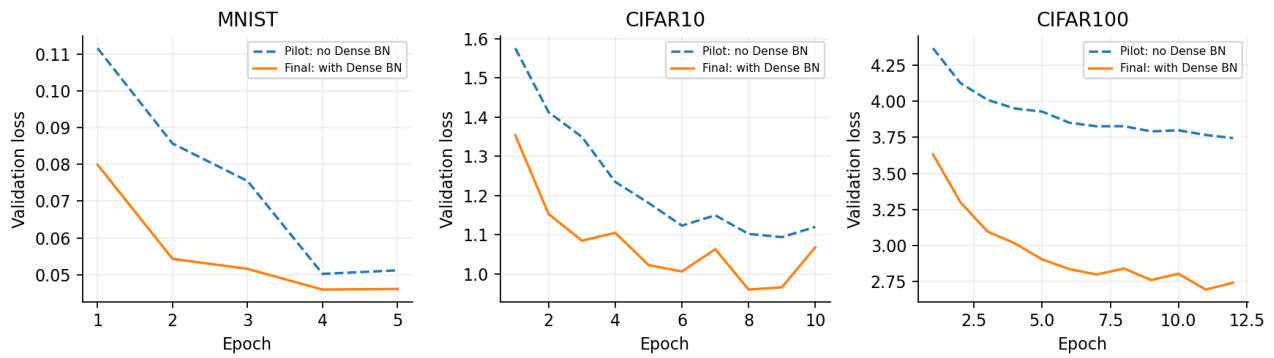
Hình 15. Accuracy test của 18 cấu hình. Cùng một thang phần trăm giúp thấy chênh lệch độ khó giữa ba dataset.

Dataset	Backend	Δ accuracy (đpt)	Train baseline (s)	Train improved (s)
MNIST	numpy	+0.23	163.4	754.7
MNIST	pytorch	+0.40	15.9	43.8
MNIST	tensorflow	+0.07	16.8	22.1
CIFAR10	numpy	+3.28	631.9	1680.2
CIFAR10	pytorch	+2.23	45.1	94.3
CIFAR10	tensorflow	+3.01	33.1	44.2
CIFAR100	numpy	+4.19	1195.8	2044.6
CIFAR100	pytorch	+3.31	56.9	113.4
CIFAR100	tensorflow	+4.00	42.6	52.0

Bảng 22. Chênh lệch improved-baseline và tổng thời gian train, chưa gồm validation/test.

Improved thêm 2.528 tham số và nhiều phép tính hơn. Lợi ích cần được cân nhắc cùng độ phức tạp và ngân sách huấn luyện. Thời gian CPU/GPU không đo trong điều kiện cô lập, nên bảng không dùng để xếp hạng tốc độ backend. Khác biệt nhỏ về accuracy cũng chưa cho thấy ưu thế ổn định khi chưa lặp nhiều seed.

5.9. Thử nghiệm ban đầu và điều chỉnh phần Dense



Hình 16. Validation loss của improved ban đầu và phiên bản cuối trong TensorFlow. Phiên bản cuối thêm BatchNorm sau Dense 64; các điều kiện huấn luyện còn lại giữ cùng cấu hình.

Dataset	Kiến trúc	Train loss cuối	Val loss tốt nhất
MNIST	Thiếu BN Dense	0,0573	0,0502
MNIST	Có BN Dense	0,0365	0,0459
CIFAR10	Thiếu BN Dense	1,1805	1,0943
CIFAR10	Có BN Dense	0,8290	0,9607
CIFAR100	Thiếu BN Dense	3,9246	3,7442
CIFAR100	Có BN Dense	2,5328	2,6943

Bảng 23. Đối chiếu train/validation của thử nghiệm kiến trúc ban đầu.

Ở CIFAR-100, bản thử có cả train và validation loss cao, phù hợp dấu hiệu underfitting trong ngân sách đã đặt. Bổ sung BN sau Dense cải thiện khả năng tối ưu của phần phân loại. Đây là quan sát thực nghiệm của cấu hình cụ thể; chưa chứng minh mọi mạng CNN đều cần BN ở Dense hoặc BN là nguyên nhân duy nhất của mọi chênh lệch.

Bốn lượt pilot và source cũ được lưu riêng. Một số test metrics của pilot đã được tạo trước khi điều chỉnh; vì vậy toàn bộ quy trình mang tính khám phá, không được trình bày như một thử nghiệm holdout hoàn toàn mù. Quyết định điều chỉnh dựa trên train/validation; bảng chính chỉ gồm kiến trúc cuối đã thống nhất cho cả ba backend.

5.10. Giới hạn và độ tin cậy của kết quả

Một seed, ngân sách nhỏ và kiến trúc gộp

Mỗi cấu hình chính chỉ chạy một seed, chưa có khoảng tin cậy giữa các lần huấn luyện. Mạng dùng 8/16 kênh, Dense 64 và 5-12 epoch để phù hợp phần NumPy từ đầu. Accuracy trên CIFAR không đại diện cho khả năng tốt nhất của CNN hiện đại. Chưa có augmentation, lịch learning rate hoặc ablation tách riêng từng thành phần.

Ảnh trùng hoàn toàn trong CIFAR-100

Backend	Mô hình	Test gốc (%)	Bỏ ảnh trùng (%)
numpy	baseline	28,23	28,23
numpy	improved	32,42	32,40
pytorch	baseline	28,84	28,83
pytorch	improved	32,15	32,14
tensorflow	baseline	28,54	28,55
tensorflow	improved	32,54	32,54

Bảng 24. Độ nhảy với 10 ảnh test có pixel đã xuất hiện trong training gốc; còn 9990 ảnh.

Đây là phép kiểm tra bổ sung theo nội dung pixel, không thay thế benchmark. Các trùng lặp train-validation vẫn có thể tác động đến chọn epoch; near-duplicate chưa được kiểm tra. Nghiên cứu nghiêm ngặt hơn cần chia nhóm ảnh trước mọi lượt thử và dùng một tập holdout mới.

Diễn giải learning curves và confidence

Train metrics được tích lũy trong khi trọng số đang đổi và dropout đang bật; validation đo sau epoch ở eval. Vì vậy validation accuracy cao hơn train accuracy không tự động là lỗi. Confidence softmax chưa được calibration, nên không dùng trực tiếp làm xác suất tin cậy cho quyết định thực tế.

CHƯƠNG 6. KẾT LUẬN VÀ TÁI LẬP

6.1. Kết luận từ bài thực hành

Bài làm hoàn thành chuỗi từ lý thuyết hợp thành hàm đến forward/backward NumPy và hai framework. Các kiểm tra gradient, đối chiếu số học và kết quả học trên dữ liệu thật cung cấp bằng chứng hỗ trợ cho tính đúng đắn của triển khai. Cùng một CNN có thể biểu diễn tương ứng qua ba hệ công cụ khi kiểm soát layout, thứ tự trọng số và trạng thái train/eval.

Trên MNIST, sáu cấu hình đạt accuracy trong khoảng 98,63% đến 99,03%. Khoảng này mô tả các lượt chạy trong bài, không phải độ biến thiên thống kê của một mô hình. Độ khó tăng rõ khi chuyển từ chữ số nền đơn giản sang ảnh tự nhiên và 100 nhãn fine.

Trên CIFAR10, sáu cấu hình đạt accuracy trong khoảng 63,01% đến 66,43%. Khoảng này mô tả các lượt chạy trong bài, không phải độ biến thiên thống kê của một mô hình. Độ khó tăng rõ khi chuyển từ chữ số nền đơn giản sang ảnh tự nhiên và 100 nhãn fine.

Trên CIFAR100, sáu cấu hình đạt accuracy trong khoảng 28,23% đến 32,54%. Khoảng này mô tả các lượt chạy trong bài, không phải độ biến thiên thống kê của một mô hình. Độ khó tăng rõ khi chuyển từ chữ số nền đơn giản sang ảnh tự nhiên và 100 nhãn fine.

Cải tiến có ích khi kết hợp với kiểm tra đường học và điều chỉnh hợp lý. Thử nghiệm ban đầu trên CIFAR-100 cho thấy thêm residual và dropout chưa đủ; chuẩn hóa phần Dense giúp bản cuối tối ưu tốt hơn trong cùng số epoch. Bài giữ cả kết quả thử chưa tốt để tránh trình bày một quá trình chỉ có thành công.

6.2. Hướng phát triển

Có thể mở rộng theo thứ tự: xây dựng split theo nhóm nội dung, lặp nhiều seed, thử augmentation chỉ trên train, dùng lịch learning rate và thực hiện ablation BN/residual/dropout. Một kiến trúc global average pooling có thể giảm phần Dense; số kênh lớn hơn có thể cải thiện biểu diễn nhưng làm NumPy tốn thời gian hơn.

Nếu giảng viên yêu cầu thêm ba dataset trong slide, cần làm một nhánh CNN 1D riêng: diabetes và nhà ở cần đánh giá giả định lân cận của cột; văn bản đánh giá quần áo cần tokenize và embedding. Không áp dụng reshape thành ảnh tùy tiện chỉ để dùng Conv2D.

6.3. Hướng dẫn chạy bằng Anaconda/Jupyter

README trong repository là hướng dẫn đầy đủ và environment.yml ghim phiên bản thư viện. Trên Windows, tạo môi trường assignment04 từ file này, kích hoạt môi trường, cài gói PyTorch cu121 nếu dùng NVIDIA GPU, rồi đăng ký kernel Python (Assignment 04). Đặt CNN_CUDA_DIR vào Library/bin của môi trường CUDA phù hợp.

Bước	Thực hiện
1. Lấy source	Clone repository hoặc giải nén gói bài nộp
2. Chuẩn bị môi trường	<code>conda env create -f environment.yml</code> ; <code>conda activate assignment04</code>
3. Dữ liệu	Đặt CNN_DATA_DIR; chạy <code>python -m src.download_data</code>
4. Mở notebook	<code>jupyter lab</code> ; chọn kernel Python (Assignment 04)
5. Đọc theo thứ tự	00 Discover → 01 Datasets → 02 NumPy → 03 PyTorch → 04 TensorFlow → 05 Compare
6. Huấn luyện lại	RETRAIN=True trong notebook hoặc <code>python run_all.py --force</code>
7. Tạo bảng/đồ thị	<code>python -m src.analyze</code>

Bảng 25. Quy trình chạy chính; các lệnh chi tiết có trong README.

Notebook mặc định đọc kết quả đã huấn luyện để người chấm xem nhanh. Ô chạy ghi rõ khi tái sử dụng kết quả; đầu ra không giả làm log huấn luyện mới. Đặt RETRAIN=True sẽ chạy thật các cấu hình và ghi đè thư mục kết quả tương ứng. Nếu cần lưu nhiều seed, nên sao lưu kết quả hoặc đổi cơ chế đặt tên trước khi chạy.

`run_all.py` chạy từng backend trong tiến trình riêng. Một cấu hình đã có `metrics.json` được bỏ qua, trừ khi dùng `--force`. Lướt bị ngắt chưa hoàn tất metric sẽ được chạy lại từ đầu; checkpoint hiện lưu trạng thái suy luận, chưa lưu đầy đủ optimizer để tiếp tục một epoch đang dở.

Nếu API Kaggle yêu cầu xác thực hoặc thay đổi, tải ba ZIP qua các link đã nêu, đặt đúng tên trong CNN_DATA_DIR rồi chạy lại bước chuẩn bị. Không cần tải dữ liệu gốc từ GitHub; repository lưu link, checksum và split, còn dữ liệu lớn được quản lý ở thư mục riêng.

6.4. Cấu trúc bài nộp và khả năng truy vết

Thư mục / file	Nội dung
notebooks/	6 notebook thuật toán/phân tích có đầu ra thực thi
src/	CNN NumPy, PyTorch, TensorFlow; tải dữ liệu, train và phân tích
tests/	Sai phân hữu hạn và đối chiếu giữa framework
results/	18 thư mục thí nghiệm; split, audit, summary và verification
figures/	Ảnh minh họa, learning curves, confusion matrix, phân tích lỗi
experiments/pilot_v1/	Mã cũ và 4 lượt thử kiến trúc ban đầu
report/	PDF hoàn chỉnh, bìa gốc và script tạo báo cáo
README.md / environment.yml	Nguồn dữ liệu, môi trường, hướng dẫn chạy

Bảng 26. Các thành phần đi kèm PDF.

Nội dung một thư mục thí nghiệm

config.json ghi dữ liệu, backend, kiến trúc, siêu tham số, phiên bản và thiết bị. history.csv có một dòng mỗi epoch. metrics.json tổng hợp test; predictions.csv giữ test_id, nhãn thật, nhãn dự đoán và confidence. test_outputs.npz giữ logits/labels để tính lại metric. confusion_matrix.csv giữ số đếm theo cặp lớp. weights lưu checkpoint có validation loss thấp nhất.

Bảng PDF làm tròn để dễ đọc; file CSV/JSON giữ độ chính xác gốc. Việc tính lại metric từ logits cho phép phát hiện chép nhầm kết quả hoặc nhầm checkpoint. Các source ZIP và mảng đã chuẩn bị có SHA-256 trong manifest; dữ liệu gốc và tài liệu sách/slide không được đưa vào repository.

Liên kết source: https://github.com/namnguyen05-vn/PTHTTM_Assignment-04. Bìa sử dụng bản PDF xuất từ mẫu Google Docs do người nộp cung cấp. Nội dung chuyên môn, số liệu và hình thực nghiệm được tạo cho Assignment 04.

TÀI LIỆU THAM KHẢO

- [1] Slide học phần Phát triển các hệ thống thông minh, intel_sys_dev_slide_04_updated_11.9.pdf, tài liệu do giảng viên cung cấp.
- [2] Deep Learning CNN Function Composition Tutorial, tài liệu hướng dẫn được cung cấp, đặc biệt phần CNN và gợi ý thực hành.
- [3] François Chollet. Deep Learning with Python, 2nd edition. Manning, 2021. Tham khảo các phần đánh giá/workflow (trang in 133-137, 160-165), CNN (202-205, 221-225) và kiến trúc hiện đại (251-262).
- [4] MNIST in CSV, Kaggle, oddrationale. [Nguồn trực tuyến](#)
- [5] CIFAR-10 Python, Kaggle, pankrzsui. [Nguồn trực tuyến](#)
- [6] CIFAR-100, Kaggle, fedesoriano. [Nguồn trực tuyến](#)
- [7] S. Ioffe và C. Szegedy. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift, 2015. [Nguồn trực tuyến](#)
- [8] K. He, X. Zhang, S. Ren và J. Sun. Deep Residual Learning for Image Recognition, 2015. [Nguồn trực tuyến](#)
- [9] N. Srivastava và cộng sự. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. JMLR 15, 2014. [Nguồn trực tuyến](#)
- [10] D. P. Kingma và J. Ba. Adam: A Method for Stochastic Optimization, 2014. [Nguồn trực tuyến](#)
- [11] K. He và cộng sự. Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification, 2015. [Nguồn trực tuyến](#)
- [12] TensorFlow. Install TensorFlow with pip; ghi chú GPU Windows bản địa. [Nguồn trực tuyến](#)
- [13] A. Krizhevsky, V. Nair và G. Hinton. CIFAR-10 and CIFAR-100 datasets. [Nguồn trực tuyến](#)
- Nguồn trực tuyến được kiểm tra ngày 13/09/2026. Báo cáo mẫu A3_05_dungvt.194.pdf chỉ được tham khảo về bố cục, không dùng làm nguồn số liệu thực nghiệm.